Deep Learning
W3: Word Embeddings, Convolutional Neural Nets and Applications

M. Vazirgiannis

Distinguished Professor, Ecole Polytechnique, France
Google Scholar: https://bit.ly/2rwmvQU
Twitter: @mvazirg

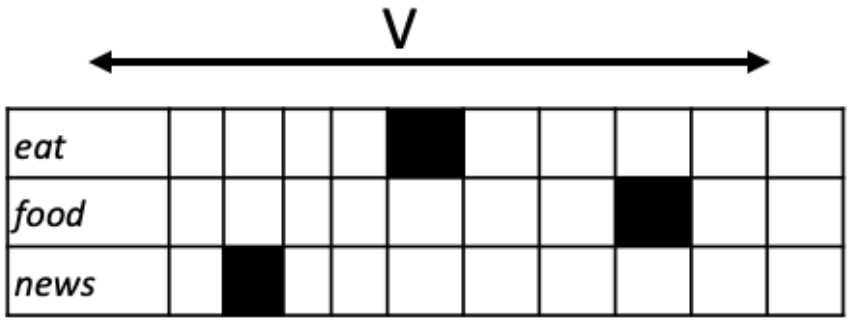
October, 2026

Outline

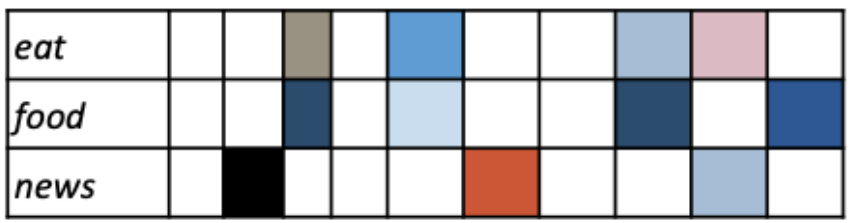
1. MLP applications to NLP
   - Word embeddings
2. Convolutional Neural Networks
3. CNN Applications to NLP

Representing Words

- One-hot vector
  - high dimensionality
  - sparse vectors
  - dimensions=$|V|$ ($10^6 < |V|$)
  - unable to capture semantic similarity between words



- Distributional vector
  - words that occur in similar contexts, tend to have similar meanings (**distributional** model of word meanings)
  - each word vector contains the frequencies of all its neighbors
  - dimensions=$|V|$
  - computational complexity for ML algorithms



Example

- We should assign similar probabilities (discover similarity) to *Obama speaks to the media in Illinois* and the *President addresses the press in Chicago*
- This does not happen because of the “one-hot” vector space representation

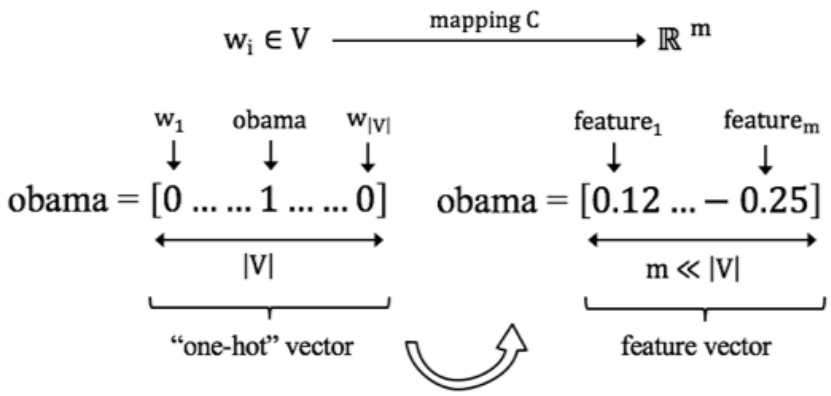
One hot

$$\text{obama} = [0\,0\,0\,0\ \ldots\ 0\,1\,0\,0] \quad \text{president} = [0\,0\,0\,1\ \ldots\ 0\,0\,0\,0] \quad \Rightarrow \overrightarrow{\text{obama}}.\overrightarrow{\text{president}} = \vec{0}$$

$$\text{speaks} = [0\,0\,1\,0\ \ldots\ 0\,0\,0\,0] \quad \text{addresses} = [0\,0\,0\,0\ \ldots\ 0\,0\,1\,0] \quad \Rightarrow \overrightarrow{\text{speaks}}.\overrightarrow{\text{addresses}} = \vec{0}$$

$$\text{illinois} = [1\,0\,0\,0\ \ldots\ 0\,0\,0\,0] \quad \text{chicago} = [0\,1\,0\,0\ \ldots\ 0\,0\,0\,0] \quad \Rightarrow \overrightarrow{\text{illinois}}.\overrightarrow{\text{chicago}} = \vec{0}$$

Word Embedding



Co-occurrence matrix

- I like to fly
- I enjoy learning to fly
- I like learning english

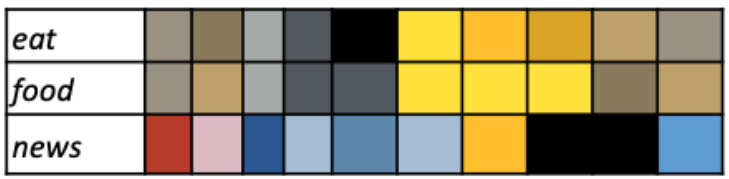
	I	like	fly	enjoy	learning	english
I		1	1	1	1	1
like	1		1		1	1
fly	1			1		
enjoy	1		1		1	
learning	1	1	1			1
english	1	1			1	

- The matrix is large and sparse
  - Storage
  - Classification
- Learn *a low dimensional dense representation of words*

Representing Words

Word embedding

- store the same contextual information in a low-dimensional vector
- **densification** (sparse to dense)
- **compression**
  - dimensionality reduction
  - dimensions=m
    $100 < m < 500$
- able to capture semantic similarity between words
- learned vectors (unsupervised)
- Learning methods
  - SVD ("standard" choice pre-2013)
  - word2vec
  - GloVe



Language model (n-gram based)

- Goal: determine $P(s = w_1 \dots w_k)$ in some domain of interest

$$P(s) = \prod_{i=1}^{k} P(w_i | w_1 \dots w_{i-1})$$

e.g. $P(w_1 w_2 w_3) = P(w_1) P(w_2 | w_1) P(w_3 | w_1 w_2)$

- Traditional n-gram language model assumption: "the probability of a word depends only on context of n-1 previous words"

$$\Rightarrow \hat{P}(s) = \prod_{i=1}^{k} P(w_i | w_{i-n+1} \dots w_{i-1})$$

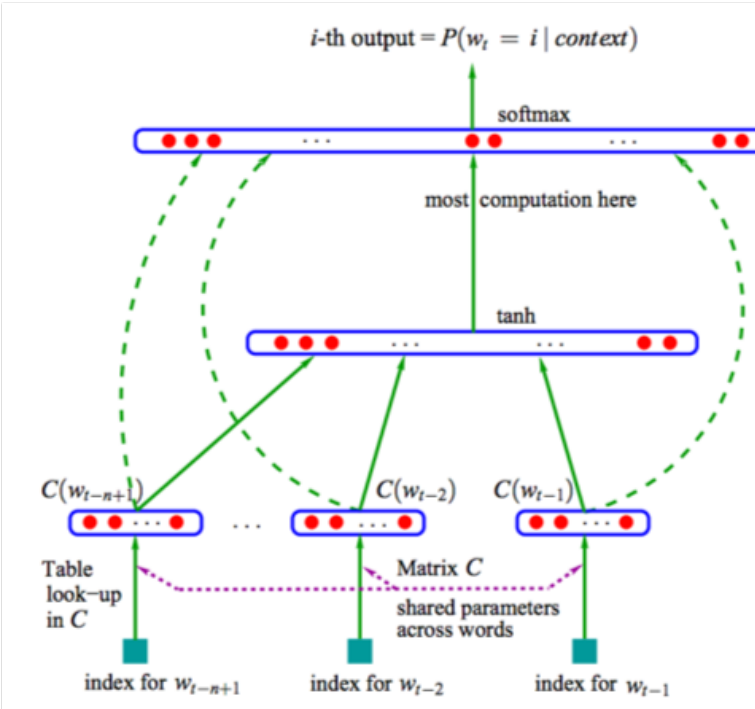
- Typical ML-smoothing learning process (e.g., Katz 1987):
  1. compute $\hat{P}(w_i | w_{i-n+1} \dots w_{i-1}) = \frac{\# w_{i-n+1} \dots w_{i-1} w_i}{\# w_{i-n+1} \dots w_{i-1}}$ on training corpus
  2. smooth to avoid zero probabilities

Learning Word Vectors

1. associate with each word in the vocabulary distributed *word feature* vector (a real-valued vector in $\mathbb{R}^m$)
2. express the joint *probability function* of word sequences in terms of the feature vectors of these words in the sequence, and
3. learn simultaneously the *word feature vectors* and the parameters of that *probability function*

- Corpus containing T training words
- Objective :
  $f(w_t, \ldots, w_{t-n+1}) = \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$
- Decomposed in two parts: $w_i \in V \xrightarrow{\text{mapping C}} \mathbb{R}^m$
  - Mapping **C** (1-hot v $\Rightarrow$ lower dimensions)
  - Mapping any **g** s.t. estimate prob $t+1|t$ previous)

  $f(w_{t-1}, \ldots, w_{t-n+1}) = g(C(w_{t-1}), \ldots, C(w_{t-n+1}))$
- C(i): i-th word feature vector (Word embedding)
- Objective function: $J = \frac{1}{T} \sum f(w_t, \ldots, w_{t-n+1})$

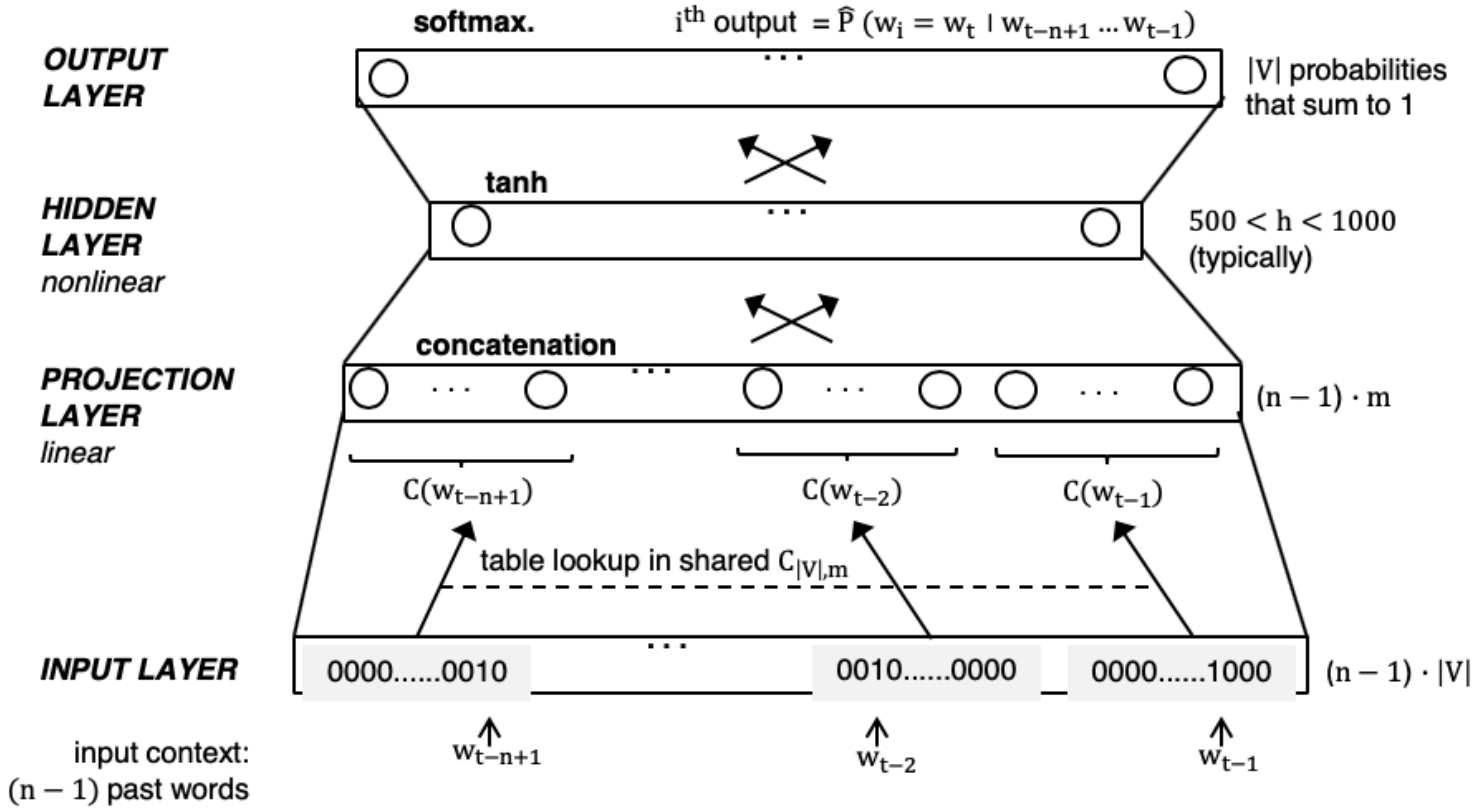


Bengio, Yoshua, et al. "A neural probabilistic language model." JMLR 3 (2003): 1137-1155.

Neural Net Language Model

For each training sequence:

- input: (context, target)
- pair: $(w_{t-n+1}, \ldots, w_{t-1}, w_t)$
- objective: minimize $E = -\log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$



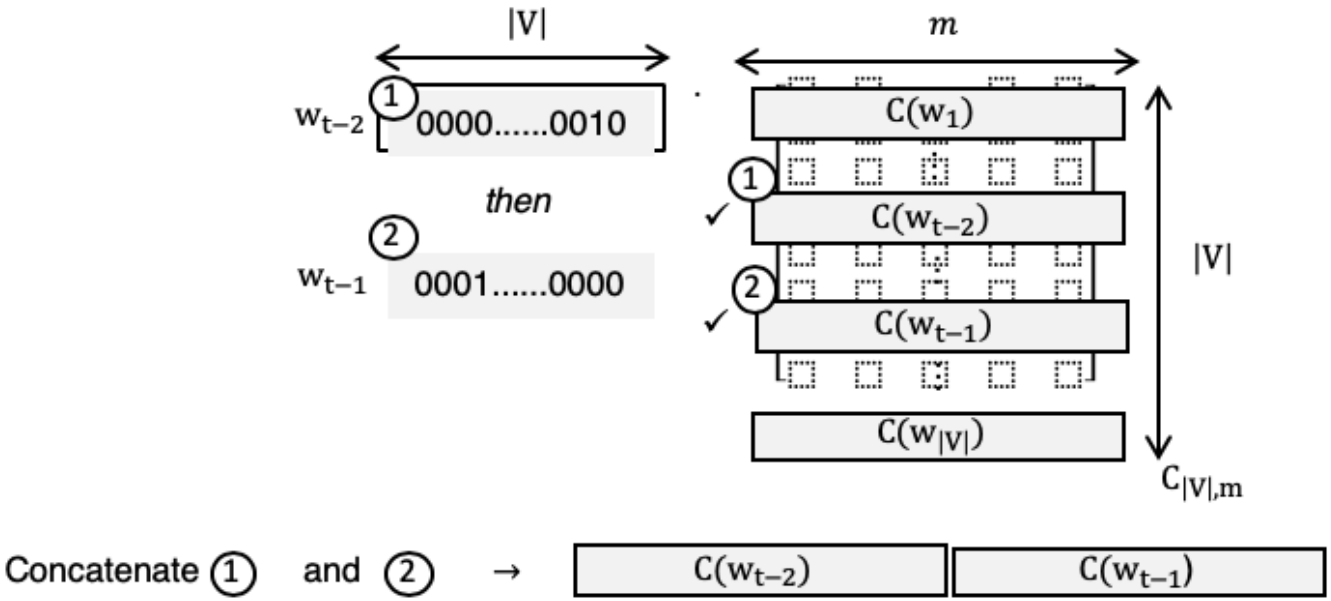
Objective function

- $E = -\log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$
- a probability between 0 and 1.
- On this support, the log is negative $\Rightarrow$ –log term positive.
- makes sense to try to minimize it.
  - Probability of word given the context be as high as possible (1 for a perfect prediction).
  - case the error is equal to 0 (global minimum).

p	log(p)	-log(p)
0.7	-0.15490196	0.15490196
0.2	-0.698970004	0.698970004

NNLM Projection layer

- Performs a simple table lookup in $C_{|V|,m}$: concatenate the rows of the shared mapping matrix $C_{|V|,m}$ corresponding to the context words. Example for a two-word context w_{t-2} w_{t-1}:



- $C_{|V|,m}$ is **critical**: it contains the weights that are tuned at each step. After training, it contains what we're interested in: the **word vectors**

Bengio, Yoshua, et al. "A neural probabilistic language model." JMLR 3 (2003): 1137-1155.

NNLM hidden/output layers and training

- Softmax (log-linear classification model) outputs positive numbers that sum to one (a multinomial probability distribution): i^{th} unit in the output layer:
  $\hat{P}(w_i = w_t | w_{t-n+1}, \ldots, w_{t-1}) = \frac{e^{yw_i}}{\sum_{i'=1}^{|V|} e^{yw_{i'}}}$

  where
  - $y = b + U \cdot \tanh(d + H \cdot x)$
  - tanh: nonlinear squashing (link) function
  - x : concatenation C(w) of the context weight vectors seen previously
  - b : output layer biases ($|V|$ elements)
  - d : hidden layer biases (h elements). Typically $500 < h < 1000$
  - U: $|V| \times h$ matrix storing the hidden-to-output weights
  - H: $h \times (n-1)m$ matrix storing the projection-to-hidden weights

  $\boldsymbol{\theta = (b, d, U, H, C)}$
  complexity per training sequence: $n * m + n * m * h + \boldsymbol{h * |V|}$
  computational bottleneck: **nonlinear hidden layer** ($h * |V|$ term)

- Training performed via stochastic gradient descent (learning rate ϵ):

$$\theta \leftarrow \theta + \epsilon \frac{\partial E}{\partial \theta} = \theta + \epsilon \frac{\partial \log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})}{\partial \theta}$$

  (weights are initialized randomly, then updated via backpropagation)

NNLM facts

- tested on Brown (1.2M words, $|V| \cong 16K$) and AP News (14M words, $|V| \cong 150K$ reduced to 18K) corpuses
- h: # hidden units, n= # words in context, m: dimensionality of word vectors.
- Brown: h=100, n=5, m=30
- AP News: h=60, n=6, m=100, **3 week** training using **40 cores**
- 24% and 8% relative improvement (resp.) over traditional smoothed n-gram LMs
- in terms of *test set perplexity*: geometric average of $1/\hat{P}(w_t|w_{t-n+1}, \ldots, w_{t-1})$
- Due to **complexity**, NNLM can't be applied to large data sets $\rightarrow$ poor performance on rare words
- Bengio et al. (2003) claims main contribution was a more accurate LM. They let the interpretation and use of the word vectors as **future work**
- On the opposite, Mikolov et al. (2013) focus on the **word vectors**

From MLPs to word vectors

Framework

- A layer: $\mathbf{h} = g(W\mathbf{x} + \mathbf{b})$
- A softmax output and a cross-entropy loss
- Gradients from backpropagation, steps from SGD

Representation Learning process

- The input is a word, encoded one-hot over a vocabulary of size V
- The task: predict a word from its neighbours
- We keep the *hidden representation*, not the predictions

A word embedding is the hidden layer of an MLP trained on a word prediction task (distributional meaning paradigm).

The first layer as a lookup table

Let $\mathbf{x}$ be one-hot: a single 1 at position i, zeros elsewhere. Then

$$W\mathbf{x} = \sum_j W_{:,j}\, x_j = W_{:,i}$$

the i-th column of W:

- The matrix product degenerates into selecting one column
- $W \in \mathbb{R}^{d \times V}$ holds one d-dimensional vector per word: it is the **embedding table**
- Backpropagation updates only the columns of the words seen in the batch, since the gradient of the others is zero

Note about dimensions:

$V \approx 10^5$ words, $d = 300$. The table holds 3×10^7 parameters, and each training example touches a handful of columns.

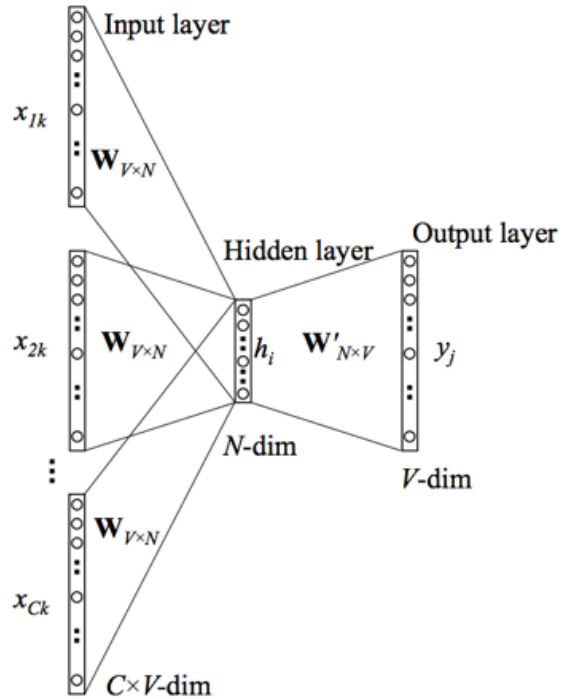
In PyTorch this layer is `nn.Embedding(V, d)`: a `nn.Linear` without bias, with the multiplication replaced by an index.

Word2Vec

- Mikolov et al. in 2013
- Key idea of word2vec: achieve better performance not by using a more complex model (i.e., with more layers), but by allowing a **simpler (shallower) model** to be trained on **much larger amounts of data**
- no hidden layer (leads to 1000 $\times$ speedup)
- projection layer is shared (not just the weight matrix) - C
- context: words from both history & future:
- Two algorithms for learning words vectors:
  - **CBOW**: from context predict target
  - **Skip-gram**: from target predict context

CBOW

- continuous bag-of-words
- continuous representations whose order is of no importance
- uses the surrounding words to predict the center word
- n-words before and after the target word

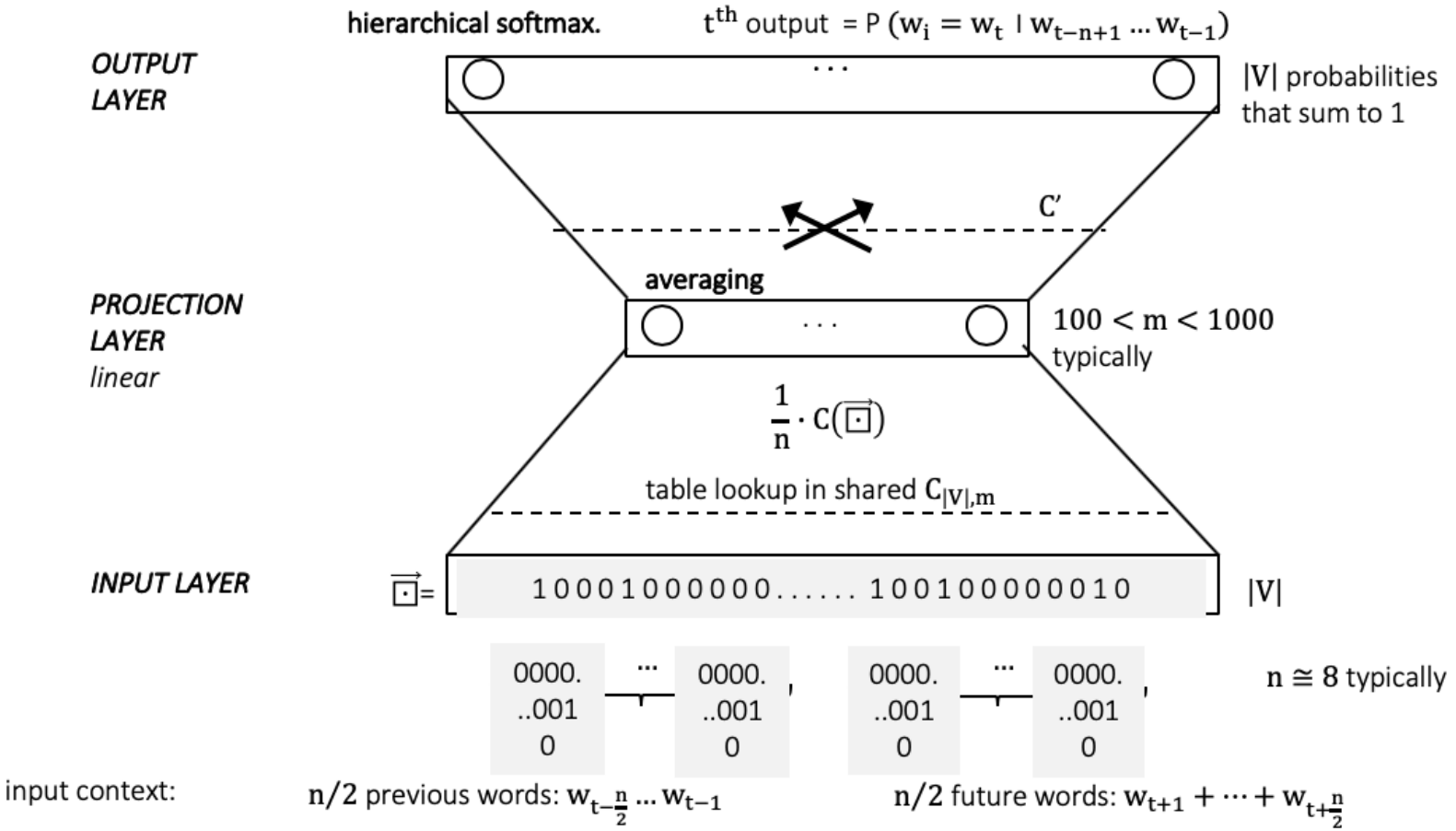


Efficient Estimation of Word Representations in Vector Space- Mikolov et al.

Continuous Bag-of-Words (CBOW)

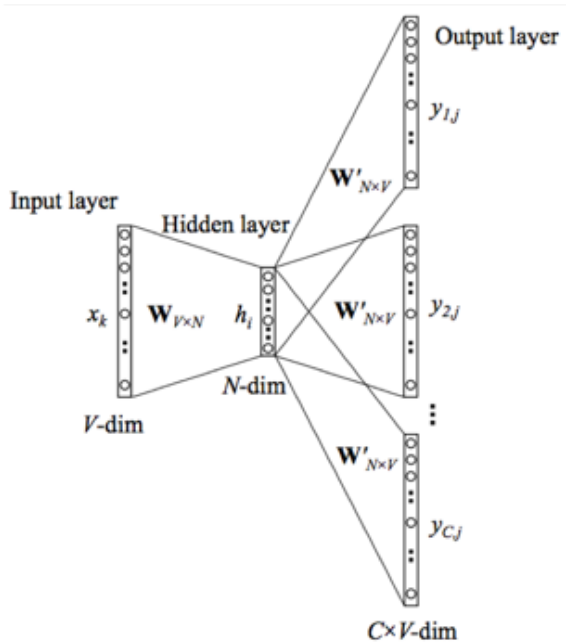
For each training sequence:

- input = (context, target)
- pair: $(w_{t-\frac{n}{2}}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+\frac{n}{2}}, w_t)$
- minimize $-\log \hat{P}(w_t | w_{t-n+1}, \ldots, w_{t-1})$



Skip-gram

- skip-gram uses, in a context, the center word to predict the surrounding words
- instead of computing the probability of the target word w_t given its previous words, we calculate the probability of the surrounding word w_{t+j} given w_t
- $P(w_{t+j}|w_t) = \frac{\exp(v_{w_t}^T v'_{w_{t+j}})}{\sum_{w \in V} \exp(v_{w_t}^T v'_{w_{t+j}})}$
- $v_{w_t}^T$ is a column of W_{VxN} and ${v'_{w_{t+j}}}^T$ is a column of W'_{NxV}
- objective function $J = \frac{1}{T} \sum_{t=1}^{T} \sum_{-n \leqslant j \leqslant n} \log p(w_{t+j}|w_t)$



Efficient Estimation of Word Representations in Vector Space- Mikolov et al. 2013

word2vec facts

- Complexity is $\boldsymbol{n * m + m * \log |V|}$ (Mikolov et al. 2013a)
- n: size of the context window (~ 10), $\boldsymbol{n \times m}$: dimensions of the projection layer, $\boldsymbol{|V|}$: size of the vocabulary
- On Google news 6B words training corpus, with $|V| \sim 10^6$:
  - CBOW with m=1000 took **2 days** to train on **140 cores**
  - Skip-gram with m=1000 took **2.5 days** on **125 cores**
  - NNLM (Bengio et al. 2003) took **14 days** on **180 cores**, for m=100 only! (m=1000 was not feasible)
- word2vec training *speed* $\cong 100K - 5M$ words/s
- Quality of the word vectors:
  - $\uparrow$ significantly with **amount of training data** and **dimension of the word vectors** (m), with diminishing relative improvements
  - measured in terms of accuracy on 20K semantic and syntactic association tasks:

Capital-Country	Past tense	Superlative	Male-Female	Opposite
Athens: **Greece**	walking: **walked**	easy: **easiest**	brother: **sister**	ethical: **unethical**

- Best NNLM: 12.3% overall accuracy. Word2vec (with Skip-gram): 53.3%

---

http://www.scribd.com/doc/285890694/NIPS-DeepLearningWorkshop-NNforText#scribd

https://code.google.com/p/word2vec/

Glove

Probability and Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k\|ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k\|steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k\|ice)/P(k\|steam)$	8.9	8.5×10^{-2}	1.36	0.96

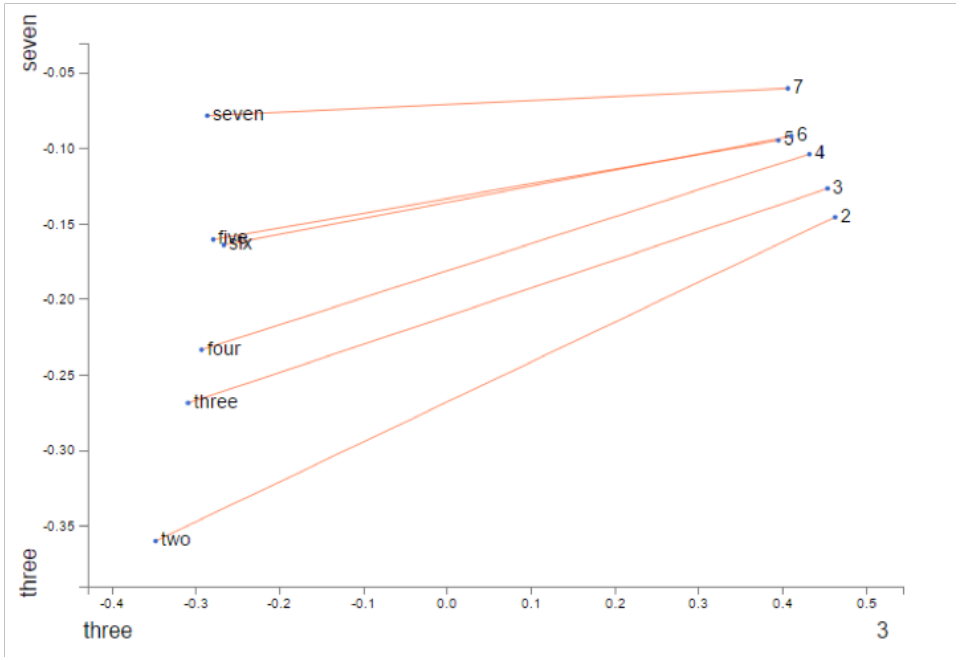
- a ratio far from 1 marks a word that discriminates between ice and steam;
- a ratio near 1 marks one that is irrelevant for this comparison, whether because it is relevant to both or to neither;
- The noise that dominates raw counts (overall word frequency) divides out.

- Ratio of co-occurrence probabilities best distinguishes relevant words
  $F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}} \Rightarrow w_i^T \tilde{w}_k + b_i + \tilde{b}_k = \log(X_{ik})$
- Cast this into a least square problem: $J = \sum_{i,j=1}^{V} f(X_{ij})(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$
- X: co-occurrence matrix
- f: weighting function $f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{if } x < x_{\max} \\ 1 & \text{otherwise} \end{cases}$
- b: bias terms
- w_i = word vector
- $\tilde{w}_j$ = context vector

https://nlp.stanford.edu/pubs/glove.pdf

Applications

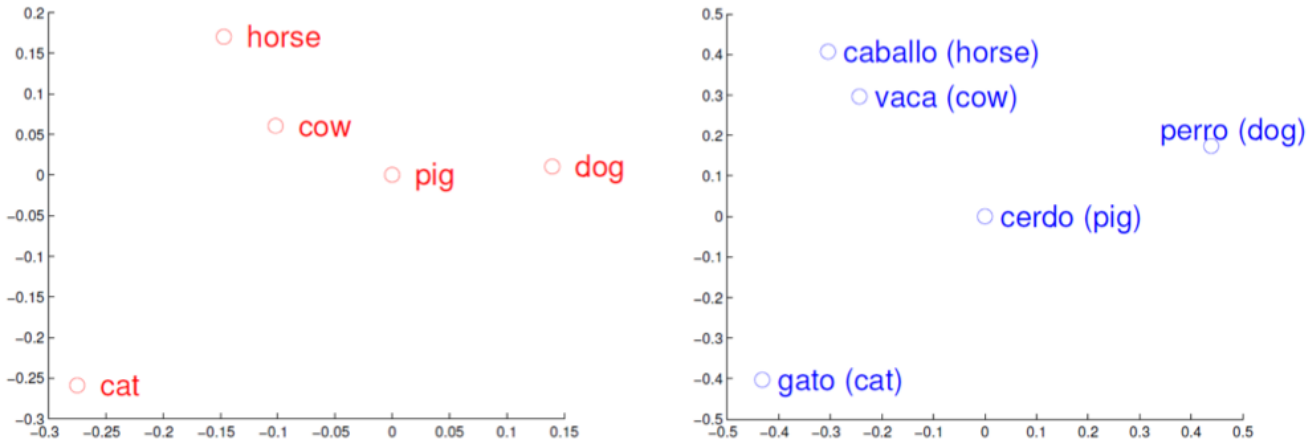
- Word analogies
- Find similar words (semantic & syntactic)
- POS tagging
- Similar analogies for different languages
- Document classification



https://lamyiowce.github.io/word2viz/

Applications

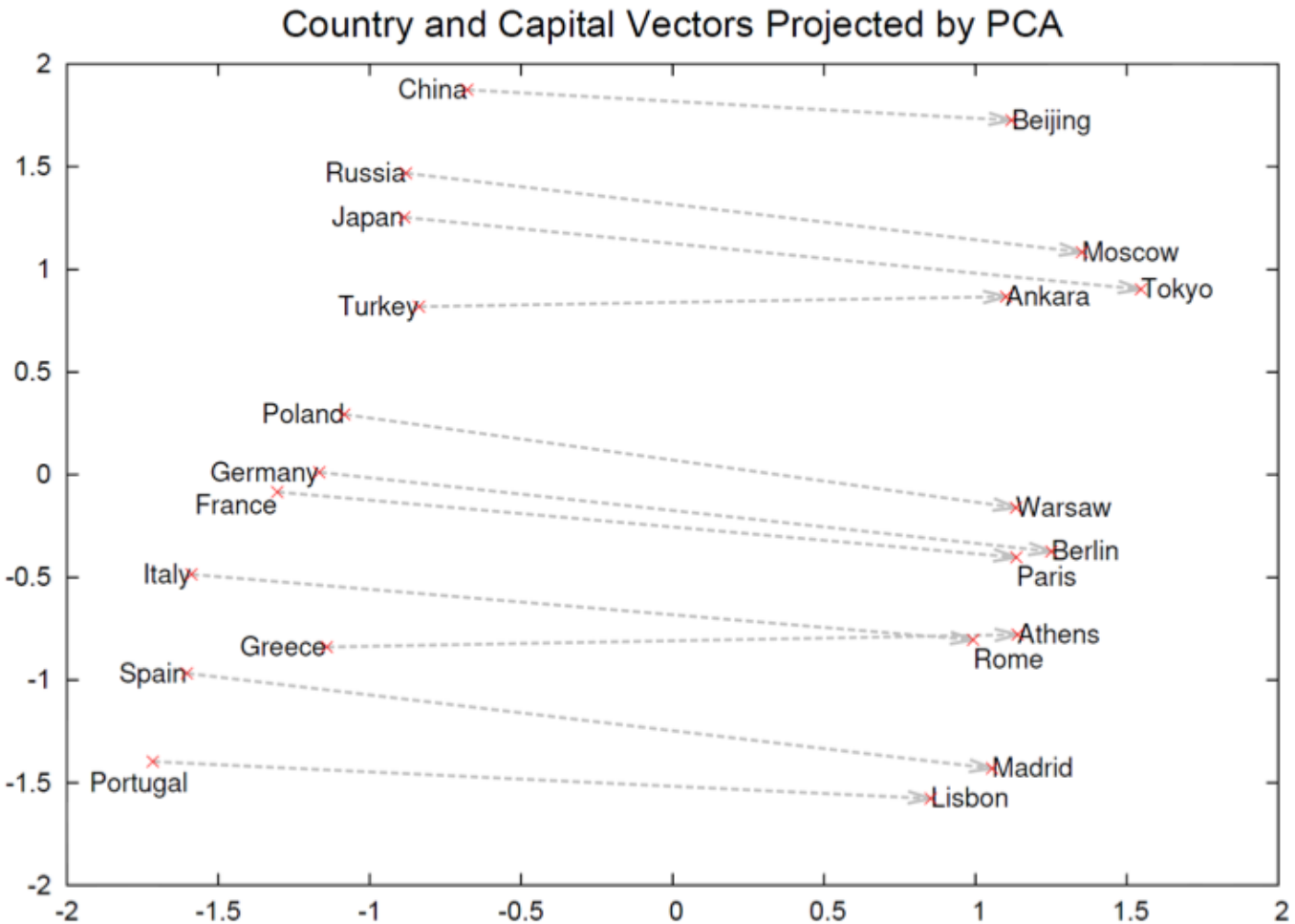
- High quality word vectors boost performance of all NLP tasks, including document classification, machine translation, information retrieval...
- Example for English to Spanish machine translation:



About 90% reported accuracy (Mikolov et al. 2013c)

Mikolov, T., Le, Q. V., & Sutskever, I. (2013). Exploiting similarities among languages for machine translation. arXiv:1309.4168.

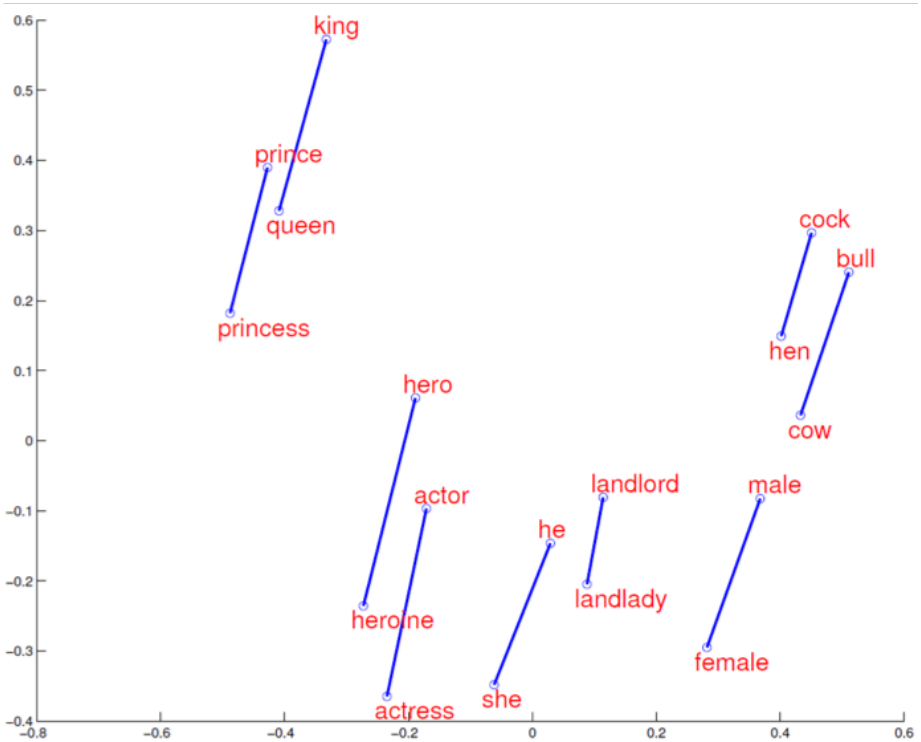
Remarkable properties of word vectors



regularities between words are encoded in difference vectors e.g., constant **country-capital** difference vector

Mikolov et al. (2013b) Distributed representations of words and phrases and their compositionality

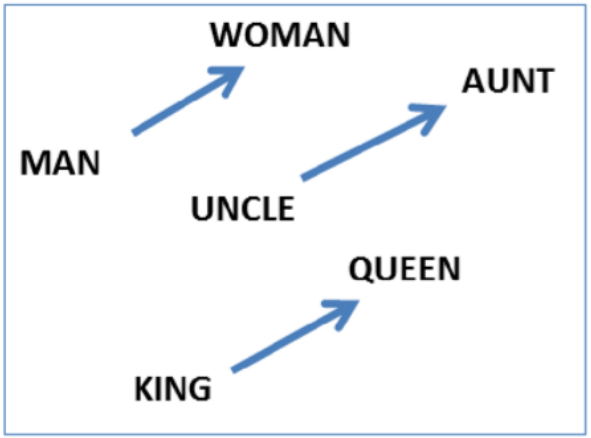
Remarkable properties of word vectors

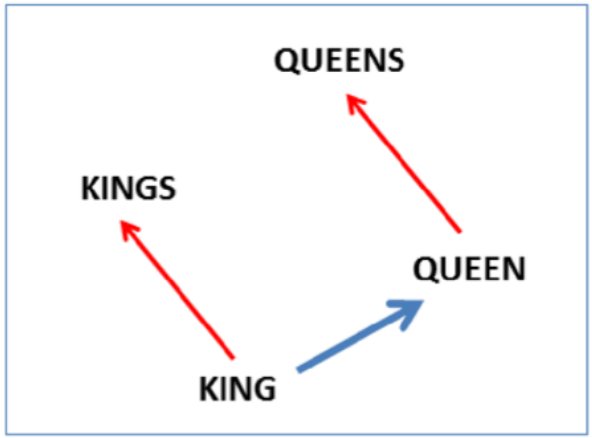


constant **female-male** difference vector

http://www.scribd.com/doc/285890694/NIPS-DeepLearningWorkshop-NNforText#scribd

Remarkable properties of word vectors





constant **female-male** difference vector

constant **singular-plural** difference vector

- Vector operations are supported and make intuitive sense:

$$w_{king} - w_{man} + w_{woman} \cong w_{queen}$$
$$w_{paris} - w_{france} + w_{italy} \cong w_{rome}$$
$$w_{windows} - w_{microsoft} + w_{google} \cong w_{android}$$
$$w_{einstein} - w_{scientist} + w_{painter} \cong w_{picasso}$$
$$w_{his} - w_{he} + w_{she} \cong w_{her}$$
$$w_{cu} - w_{copper} + w_{gold} \cong w_{au}$$

- Original tutorial by T.Mikolov

https://projector.tensorflow.org/

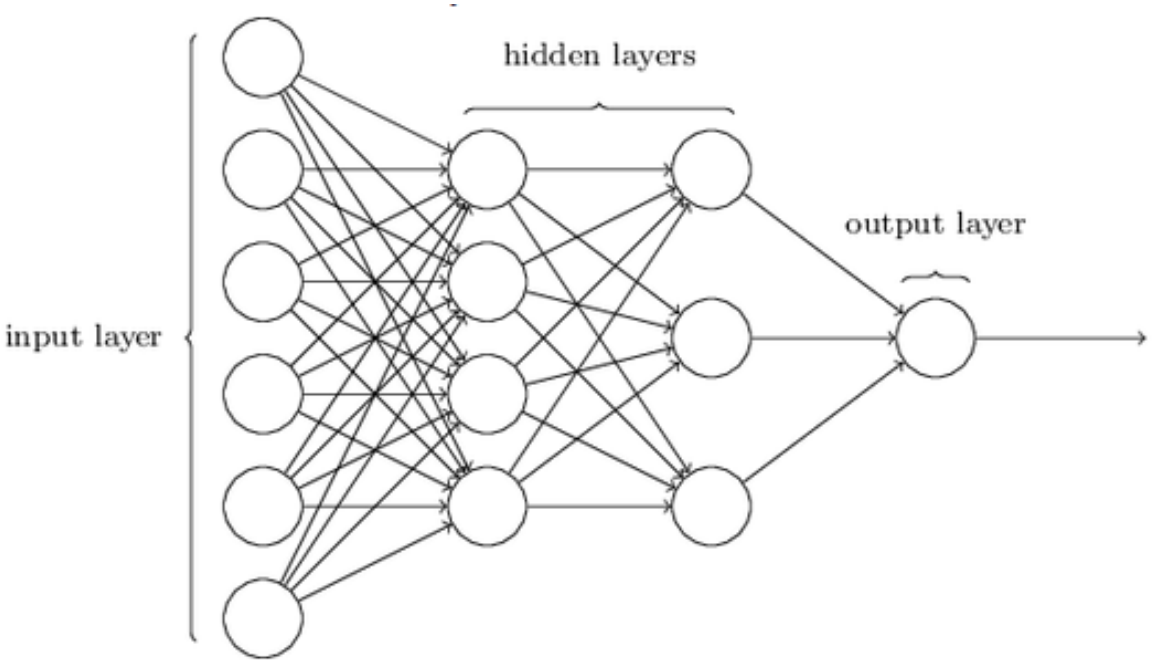
Outline

1 MLP applications to NLP
- Word embeddings

2 Convolutional Neural Networks

3 CNN Applications to NLP

NNs architecture - Multilayer perceptron (MLP)



- i.e. case of 64x64 greyscale image representing a single number (i.e. "8")
- MLP with (64x64) 4096 input neurons
- A single neuron as output ($>$ 0.5: “is 8” , $<$ 0.5 not "8”)

Convolutional Neural Networks: from pixels to features

- A Neural Network learns: a loss, a gradient, backpropagation.
- Mostly applied to to sequences of tokens.
- *Let's apply it to a grid of pixels* —
- A dense layer treats an image (224 pixels x 224 pixels x 3 channels)as a bag of 150,528 unrelated numbers.
- does not know that two pixels are neighbours
- assume an MLP to encode image into a hidden layer of 64 units: $(224 \cdot 224 \cdot 3) \cdot 64 + 64 = 9{,}633{,}856$ parameters





- One 3×3 filter — nine numbers — sweeps across the whole image.
- recognizes every vertical edge in it.

Three assumptions behind CNNs design

} PRIOR I -- LOCALITY

Neighbours matter, distant pixels don't

A neuron looks at a small image patch, not the whole image. Every weight outside that patch is fixed at zero.

Consequence: features built from pixels physically near each other — thus edges, corners and textures defined.

}PRIOR II -- WEIGHT SHARING

A feature is a feature anywhere

Kernel/feature (3x3): same 9 weights reused at every spatial position => one filter, not one per location.

Consequence: the parameter count collapses, every occurrence of a pattern anywhere contributes gradient to the same 9 numbers.

}PRIOR III -- EQUIVARIANCE

Shift the input, the output shifts too

Formally $\mathrm{conv}(\mathrm{shift}(x)) = \mathrm{shift}(\mathrm{conv}(x))$.

Consequence: network never learns "the same thing, another location". Pooling later turns this equivariance into partial *invariance*.

Convolutional Neural Networks

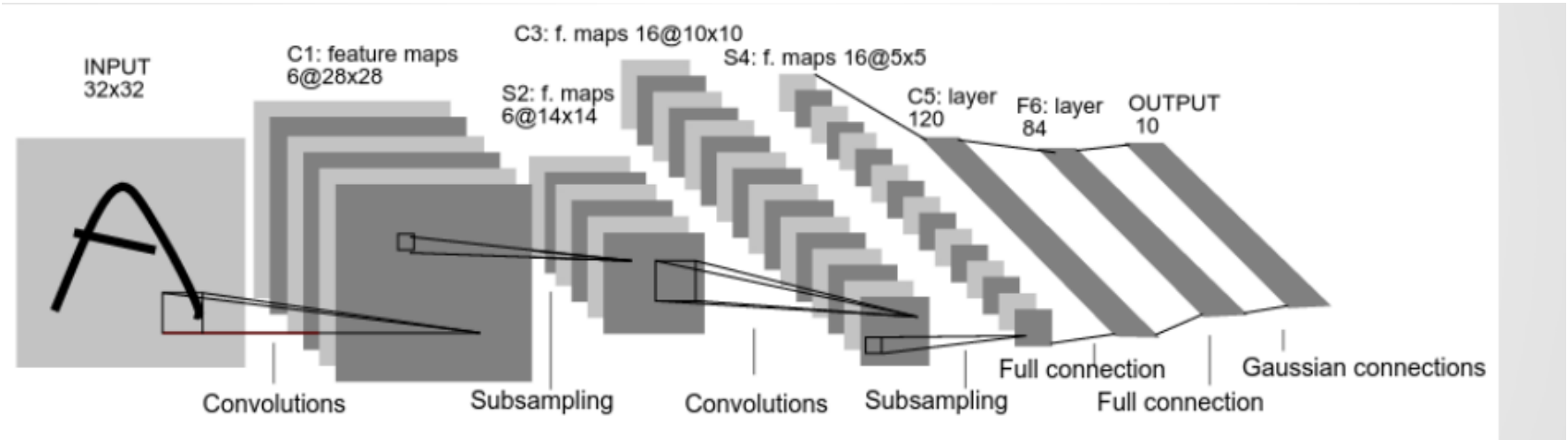
- inspired by studies of the cat's visual cortex [1], developed in computer vision to work on regular grids such as images [2].
- Feed-forward NNs , each neuron receives input from a neighborhood of the neurons (receptive fields) in the previous layer.
- Receptive fields, allow CNNs to recognize complex patterns in a hierarchical way, by combining lower-level, elementary features into higher-level features compositionality.
  - raw pixels $\Rightarrow$ edges$\Rightarrow$ shapes $\Rightarrow$ objects.
- absolute positions of features in the image are not important – only useful respective positions are useful to compose higher-level patterns.
- Model detect a feature regardless of its position in the image - local invariance.
- Compositionality, local invariance two key concepts of CNNs.

---

[1] Hubel, David H., and Torsten N. Wiesel (1962). Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. The Journal of physiology 160.1:106-154. 4

[2] LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324. 3, 4

The first CNN architecture



Lenet-5 (Lecun-98), Convolutional Neural Network for digits recognition

Kernel / filter	learned weight matrix
Receptive field	input region an output pixel depends on
Feature map	the full 2-D output of one filter
Channel	one feature map in a stack of them

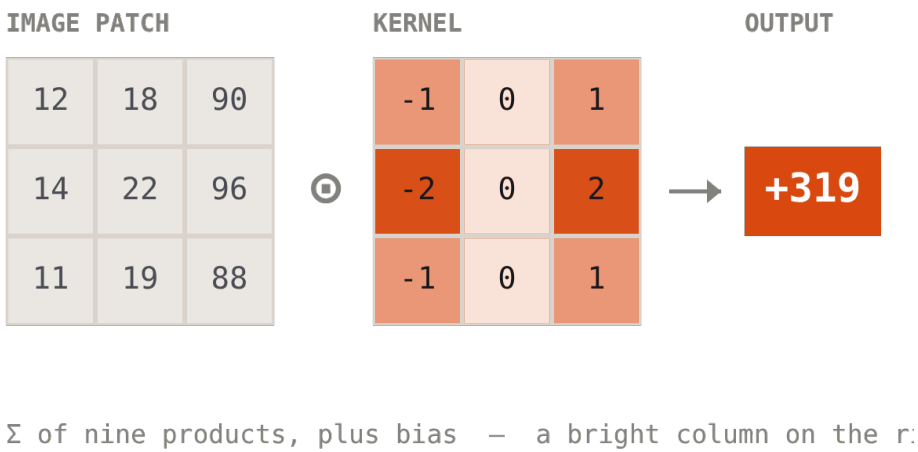
Convolution: what does it compute?

Assume K — the **kernel**. Place it over a patch. Multiply elementwise each weight by the pixel value, sum the products, add a bias: one pixel of the output. Slide over one position and Repeat.

$$y[i,j] \;=\; b + \sum_u \sum_v K[u,v]\; x[i+u,\, j+v]$$

output large when patch *resembles* the kernel — i.e. unnormalised correlation. Therefore: kernel is **template**, and the *feature map* is a map of where that template matches.

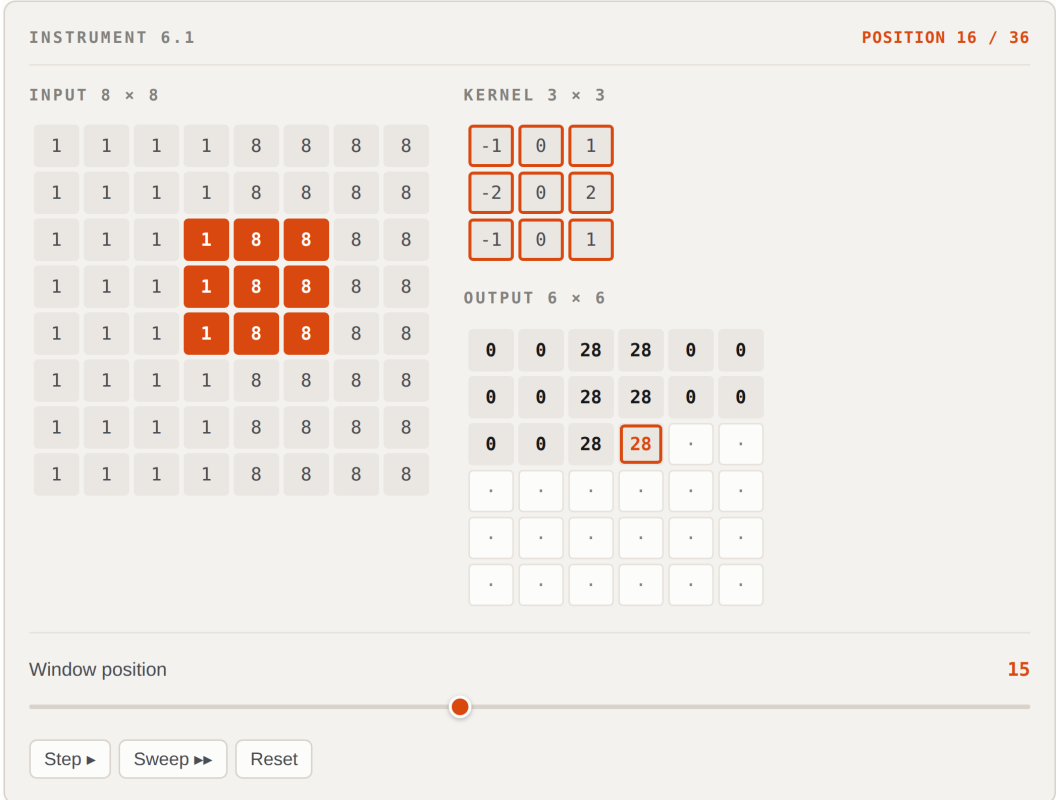
The kernel is learned



Convolution - an example

An 8×8 patch with a vertical edge, convolved with Sobel kernel: sits at one position; nine products summed into one output pixel; the kernel then moves on.

output size 6×6: 3×3: kernel matrix fits in six positions along an eight-pixel row/column



What to notice

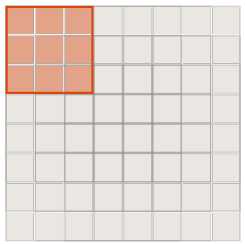
This kernel "recognizes" a *vertical* edge, returns almost zero on horizontal ones. It is detecting an edge *at one orientation*. That is why a layer needs many filters.

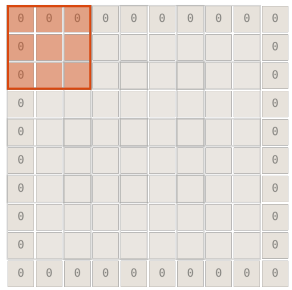
Stride, padding

$$\text{out} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

- **Stride** s — how far the window jumps. $s = 2$ halves the output size ;
- **Padding** p — zeros around the border so the window can sit on edge pixels. $p = (k-1)/2$ keeps the output the same size as the input: “same” padding.

Without padding: every layer shrinks image=> border pixels visited less often than central ones=> almost every modern architecture pads.





n=8, k=3, s=1, p=0 $\Rightarrow 6 \times 6$

n=8, k=3, s=2, p=1 $\Rightarrow 4 \times 4$

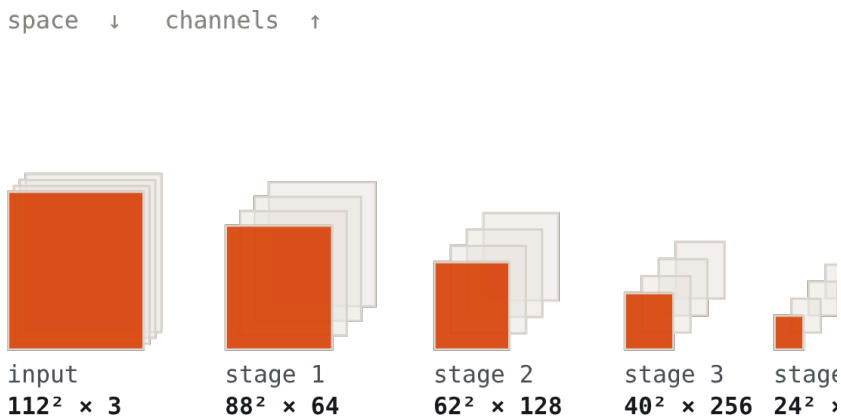
Channels

Almost every diagram shows a 2-D square sliding over a 2-D square. Real convolution is 3-D: the kernel spans *all* input channels at once.

$$\underbrace{k \cdot k \cdot C_{\text{in}} \cdot C_{\text{out}}}_{\text{weights}} + C_{\text{out}}$$

ResNet stage 2, 3×3**,** $64 \to 128$**:**
$3 \cdot 3 \cdot 64 \cdot 128 + 128 = 73{,}856$**.**

- **Each filter collapses depth.** A $3 \times 3 \times 64$ filter produces one 2-D map, not 64 — the sum runs over channels too.
- **The stack is the representation.** With 128 filters, each position holds a 128-dimensional vector describing what is there.



FM size vs channels: trade-off

Early layers: lot of space and few channels: many positions, each described crudely. Late layers: little space and many channels: few positions, each described richly - a picture => a description.

Backpropagation through a convolution

A convolution is a sum of products, so the rules for MLPs still apply. The only new fact is that one weight is used at *many* positions.

Three gradients, one backward pass

- Kernel: $\dfrac{\partial C}{\partial K[u,v]} = \sum_{i,j} \delta[i,j]\; x[i{+}u,\; j{+}v]$
- Bias: $\dfrac{\partial C}{\partial b} = \sum_{i,j} \delta[i,j]$
- Input (to pass further back): δ convolved with the kernel flipped in both directions

- (u, v) index a position inside the kernel.
- (i, j) index a position in the output feature map. $y[i, j]$ is one output pixel, and
- $\delta[i,j] = \dfrac{\partial C}{\partial y[i,j]}$ is the error arriving there.

Exercise: one filter, by hand

Input X (5×5), one bright vertical stripe; kernel K (3×3), bias $b = 0$; stride 1, no padding.

$$X = \begin{bmatrix} 0 & 0 & 10 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 \end{bmatrix}$$

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

$$y[i,j] = b + \sum_{u,v} K[u,v]\, x[i+u,\ j+v]$$

(a) What is the size of the output, and what are its values?

(b) Give the output size for (k, s, p) equal to $(3, 1, 1)$, $(3, 2, 1)$ and $(5, 1, 0)$.

(c) The loss gradient arriving at the output is $\delta = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$. Compute $\partial C/\partial K$ and $\partial C/\partial b$.

Exercise: solution

(a) Forward pass $(5-3)/1+1=3$, so 3×3. All rows of X are equal, so all rows of y are equal:

$$y=\begin{bmatrix}-30 & 0 & 30\\ -30 & 0 & 30\\ -30 & 0 & 30\end{bmatrix}$$

j=0: $-1\cdot 10$ per row $\Rightarrow -30$. j=1: the stripe falls on the zero column $\Rightarrow 0$. j=2: $+1\cdot 10$ per row $\Rightarrow +30$.

(b) Geometry $\lfloor (n+2p-k)/s\rfloor+1$:

k	s	p	out	
3	1	1	5×5	(“same”)
3	2	1	3×3	
5	1	0	1×1	

(c) Backward pass Only two entries of δ are non-zero, so each weight collects two terms, $x[u,v]$ and $x[u{+}2,v{+}2]$:

$$\frac{\partial C}{\partial K}=\begin{bmatrix}10 & 0 & 10\\ 10 & 0 & 10\\ 10 & 0 & 10\end{bmatrix},\qquad \frac{\partial C}{\partial b}=2$$

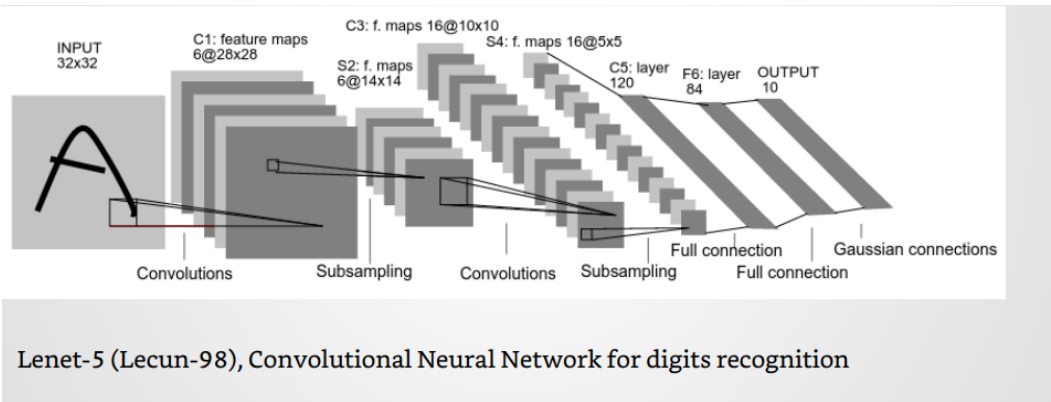
What to take away

Zero response *on* the stripe, opposite signs on either side: the filter detects a transition, and its sign gives the direction. A horizontal stripe would give zero everywhere, which is why one layer needs many filters.

Feature maps

- Feature Map - Obtained by convolution of the feature matrix with a linear filter, adding a bias term and applying a non-linear function
  - Non-linear functions
    - Sigmoid $\frac{1}{1+e^{-x}}$
    - Tanh $\frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$
    - Rectified Linear Unit (ReLU) $\rightarrow$ Most popular choice avoids saturation issues, makes learning faster

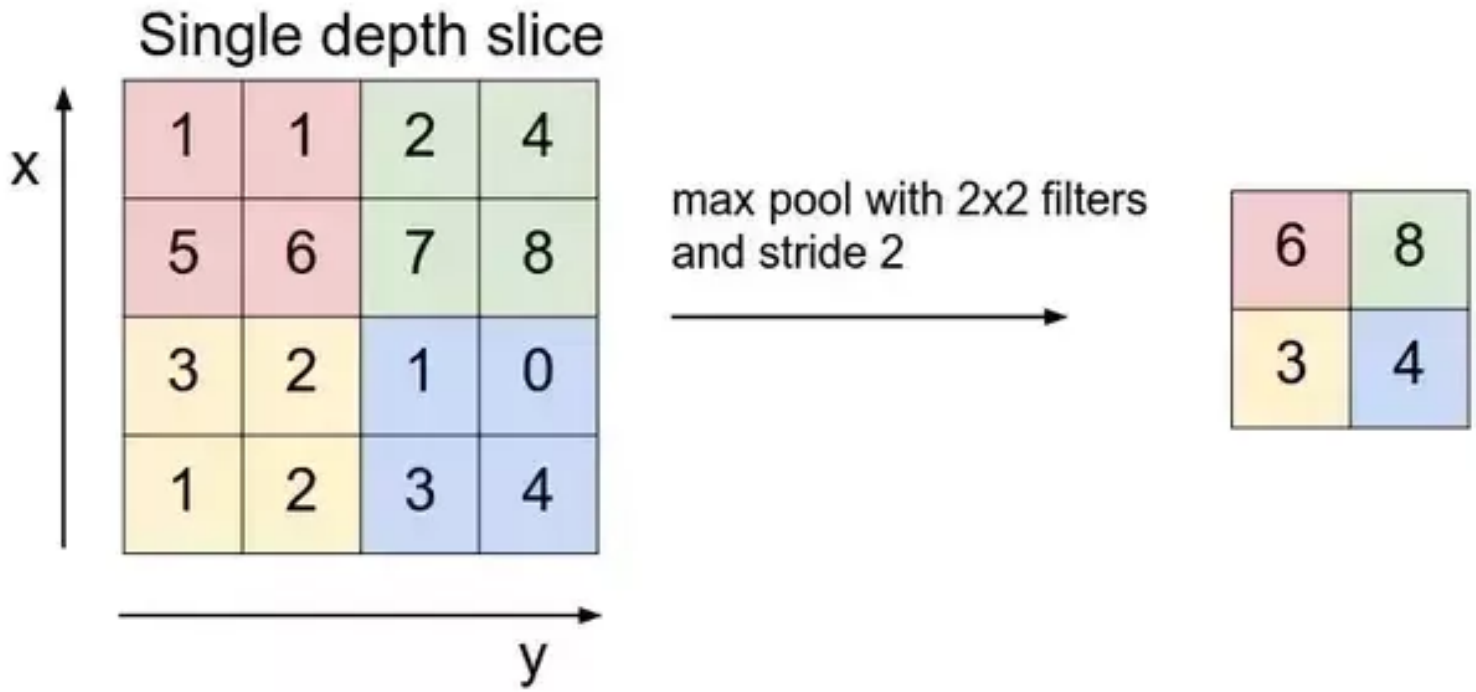
$$f(x) = max(0, x)$$



Lenet-5 (Lecun-98), Convolutional Neural Network for digits recognition

- Require a number of such feature maps at each layer to capture sufficient features

Max pooling - Example



See demo: https://poloclub.github.io/cnn-explainer/

Pooling: blurring position on purpose

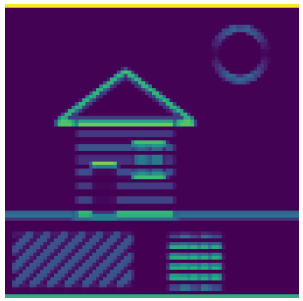
Pooling replaces each feature map window with a single summary — maximum or mean.
No parameters neither learnable.
Answers question the conv layer cannot: *does this feature appear near here*, rather than *at exactly this pixel*.

$$\text{out} = \left\lfloor \frac{n-k}{s} \right\rfloor + 1$$

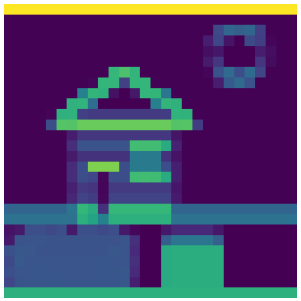
- **Max pooling** keeps the strongest evidence and discards where in the window it came from — converting equivariance into local invariance.
- **Average pooling** keeps everything but blurs. Rare inside modern nets, universal at the very end as *global average pooling*.



IN, 166×166



MAX 2×2, 83^2



MAX 6×6, 28^2

Max 2×2 discards 75% of the values and the scene is still perfectly legible. Max 6×6 discards 97% — and the structure of the house still survives. That robustness is the point; it is also why pooling was so easy to over-use.

Stacking small kernels

- VGG is a convolutional architecture from Simonyan and Zisserman (Visual Geometry Group, Oxford), published at ICLR 2015
- every convolution is 3×3 with stride 1 and padding 1, every pooling is 2×2 max-pool, and the channel count doubles after each pooling stage (64, 128, 256, 512, 512)

Two 3×3 layers in a row see a 5×5 patch of the input; three see 7×7. In general the receptive field grows as $1 + L(k - 1)$ for stride 1.

	Field	Weights	Nonlin.
One 7×7	7×7	$49\,C^2$	1
Three 3×3	7×7	$27\,C^2$	3

Same field of view, 45% fewer weights and three nonlinearities instead of one.

VGG vs. AlexNet

AlexNet had used 11×11 and 5×5 kernels. VGG showed that a stack of 3×3 layers is strictly better

Stride and pooling enlarge the receptive field much faster: each halving of the resolution doubles what one output pixel covers.

Number of parameters

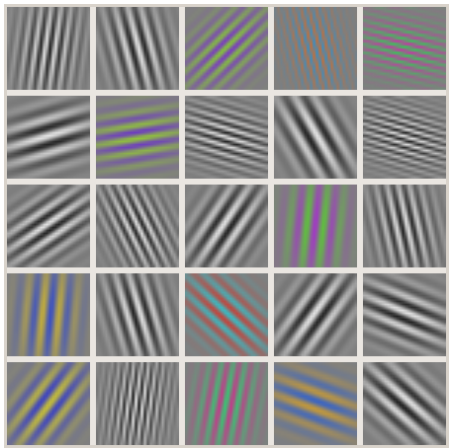
A small network on 32×32 RGB images, ten classes. "same" padding, 2×2 max pooling.

Layer	Output	Parameters	Multiply-adds
Conv 3×3, $3 \to 16$	$16 \times 32 \times 32$	448	0.46 M
Max-pool 2×2	$16 \times 16 \times 16$	0	—
Conv 3×3, $16 \to 32$	$32 \times 16 \times 16$	4,640	1.18 M
Max-pool 2×2	$32 \times 8 \times 8$	0	—
Conv 3×3, $32 \to 64$	$64 \times 8 \times 8$	18,496	1.18 M
Max-pool 2×2	$64 \times 4 \times 4$	0	—
Flatten + Dense $1024 \to 10$	10	10,250	0.01 M
Total		33,834	2.8 M

Two observations. The convolutions hold the *compute* but few parameters; a single dense layer holds almost a third of the *parameters* while doing nothing. Replace it with global average pooling and the count drops to 650. In VGG-16, the dense head alone is 102 M of the 138 M.

Features hierarchy learned

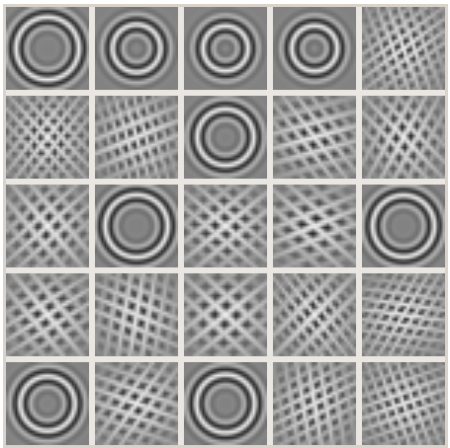
Train a CNN on ImageNet with a classification loss, then visualise what each layer responds to. The same four-stage progression appears — across architectures, across datasets, across research groups.



}CONV 1 -- 3-7 PX

Oriented edges, colour opponency

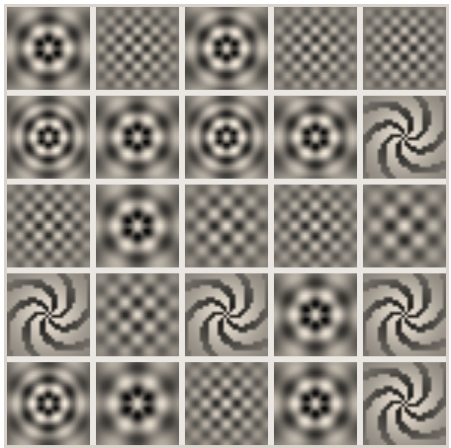
Gabor-like bars at every orientation and frequency. Strikingly similar to simple cells in mammalian V1.



}CONV 2 -- 10-20 PX

Corners, curves, junctions

Pairs and triples of edges combined. The first features with a shape rather than just a direction.



}CONV 3-4 -- 30-80 PX

Parts and repeated motifs

Wheels, eyes, honeycomb, fur. These fire on the same part across very different categories.



}CONV 5+ -- >150 PX

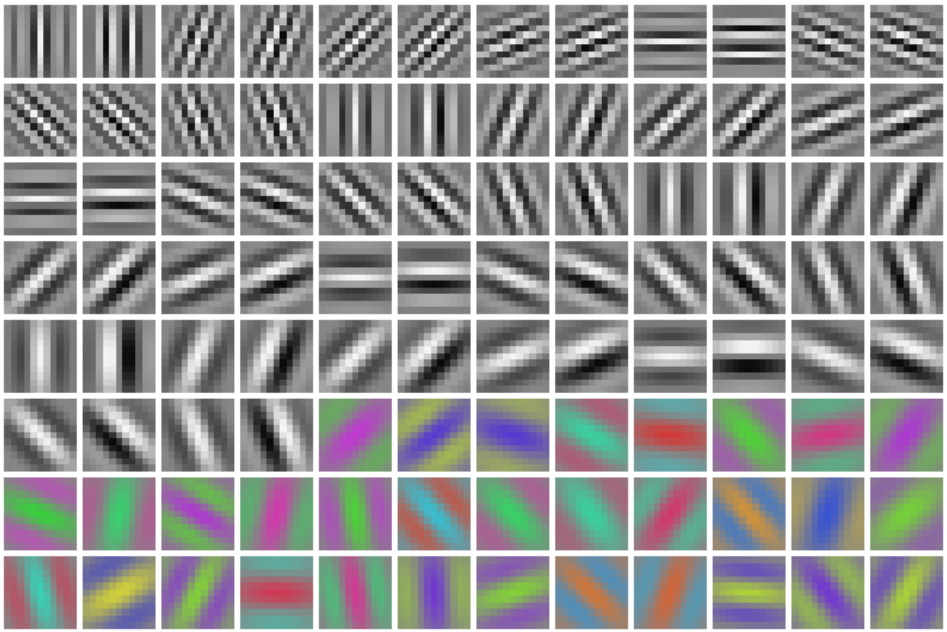
Whole objects and scenes

Faces, dogs, keyboards. Units become reliable enough to act as detectors for a category.

Why this matters

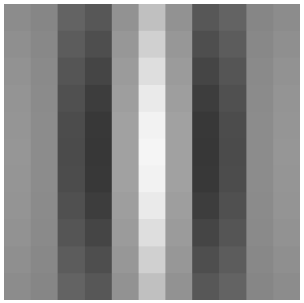
Early layers learn **generic** features building on the ones of the previous layer: an edge detector is an edge detector whether you classify dogs or diagnose tumours. That is the basis of **transfer learning**.

A first-layer filter map features

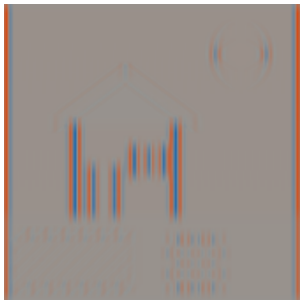


96 filters — the size and composition of AlexNet's conv1. Roughly two thirds are oriented grey Gabors at four scales; the rest are low-frequency colour-opponent blobs.

Each filter is nearly blind to everything except edges at its own orientation and scale. 96 of them together leave almost nothing in the image unmeasured — which is precisely what a *basis* means.



ONE FILTER



ITS RESPONSE

Transfer learning: reusing the hierarchy

The first layers learn edges and textures whatever the task, so they need not be learned again.

Your data	Similar to the source	Different from the source
Small	Freeze everything, retrain the head	Freeze the early layers, retrain the later ones
Large	Fine-tune the whole network, small lr	Fine-tune everything, or train from scratch

- Replace the classification head: the body gives features, only the last layer knows the classes
- Fine-tune with a learning rate 10 to 100 times smaller, otherwise the first updates destroy the pretrained features
- A few thousand images are often enough, against the million the source model needed

This, rather than training from scratch, is what most applied computer vision consists of.

In PyTorch

```
nn.Conv2d(in_channels, out_channels, kernel_size,
          stride=1, padding=0)
```

- Tensors are **(N, C, H, W)**: batch, channels, height, width
- Weight shape: $(C_{\text{out}}, C_{\text{in}}, k, k)$, exactly the formula on the channels slide
- `padding='same'` keeps the spatial size for odd k and stride 1
- The library computes cross-correlation, without flipping the kernel; since K is learned, the flip is absorbed into the weights

Shape errors

A missing `unsqueeze` for the batch dimension; an image in (H, W, C) order from NumPy; and a dense head whose input size no longer matches after changing a stride. Print `x.shape` after each layer once to debug shape problems.

Limits of convolutional NNs

Equivariance holds for *translation* only. Nothing in the architecture provides the rest.

Transformation	**In the architecture?**	**Usual remedy**
Translation	Yes, by construction	—
Rotation	No	Augmentation; equivariant architectures
Scale	No	Augmentation; multi-scale features
Illumination, colour	No	Normalization; colour jitter
Occlusion	No	Augmentation (random erasing, cutout)

- **Data augmentation** is how an invariance that is not in the wiring gets bought with examples:
- every image is randomly transformed before it enters the network: flips, rotations of a few degrees, random crops and rescalings, colour jitter...

CNN versus MLP

Property	Convolutional	Dense / MLP
Parameters, first layer	1,792 (3x3x3x64+64) — independent of image size	9,633,856 ((224·224·3)·64+ 64) — scales with area
Spatial structure	Built in: locality + sharing	Destroyed by flattening
Translated input	Output translates with it, free	Learned position by position
Test-time input size	Anything, if the head is global-pooled	Fixed at build time
Cost driver	Compute	Memory
ImageNet-1k top-1	**76–84%** (ResNet-50 → ConvNeXt-L)	Well under 50%

Conclusion

parameter count is not the deepest advantage. An MLP with enough width and data *can* represent anything a CNN can: universal approximation guarantees it.

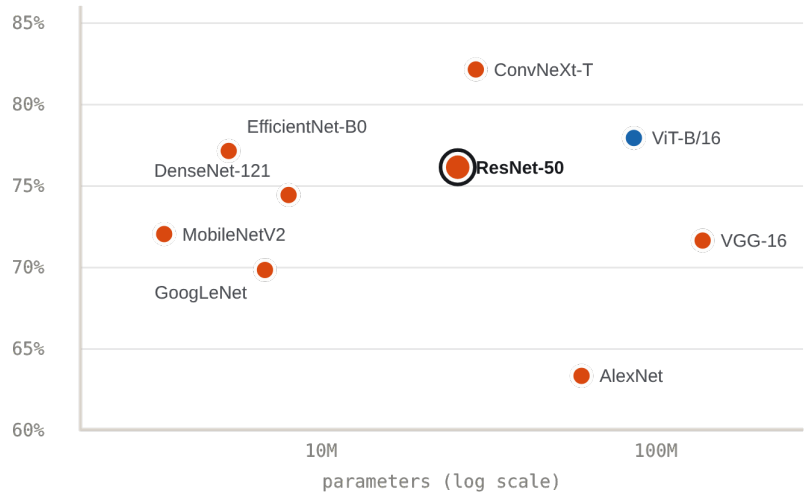
BUT "enough data" means seeing every feature at every position in every combination. The CNN gets translation free, from architecture.

In conclusion: a CNN is not a more *powerful* model than an MLP. It is a more *constrained* one, where the constraints are optimal for images.

CNN-Architentures- Twenty-four years in ten models

Year	Model	Params	Depth	Top-1
1998	LeNet-5	60 K	5	—
2012	AlexNet	60 M	8	63.3%
2014	VGG-16	138 M	16	71.6%
2014	GoogLeNet	6.8 M	22	69.8%
2015	**ResNet-50**	**25.6 M**	**50**	**76.1%**
2017	DenseNet-121	8.0 M	121	74.4%
2018	MobileNetV2	3.4 M	53	72.0%
2019	EfficientNet-B0	5.3 M	82	77.1%
2020	ViT-B/16	86 M	12	77.9%
2022	ConvNeXt-T	29 M	36	82.1%

AlexNet 2012 cut ImageNet top-5 error from 26.2% to 15.3% in one year — the result that pivoted the field. **GoogLeNet** was twenty times smaller than VGG and more accurate. **ViT-B/16** is shown at its ImageNet-1k-only figure; with JFT-300M pre-training it reaches roughly 84% , which is the whole argument of the previous slide.



ImageNet-1k top-1, standard single-crop evaluation; horizontal axis is logarithmic. LeNet is omitted — it was never evaluated on ImageNet. Note that accuracy is not a function of size: GoogLeNet beats nothing by being small, and VGG-16 buys very little with its 138 million parameters.

Outline

1 MLP applications to NLP
- Word embeddings

2 Convolutional Neural Networks

3 CNN Applications to NLP

CNN for Text Classification

- Use the word embeddings of the document terms as input for Convolutional Neural Network
- Input must be fixed size
- Applies multiple filters to concatenated word vectors
- Produces new features for every filter
- picks the max as a feature for the CNN

CNN for text classification

- Use the high quality embeddings as input for Convolutional Neural Network
- Applies multiple filters to concatenated word vector

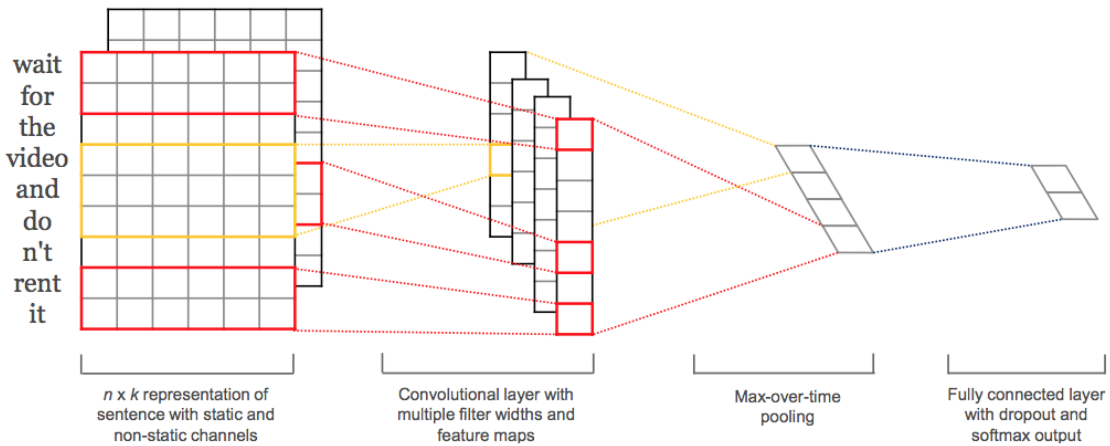
$$\mathbf{x}_{1:n} = \mathbf{x}_1 \oplus \mathbf{x}_2 \ldots \mathbf{x}_n$$

- Produces new features for every filter

$$c_i = f(\mathbf{w} \cdot \mathbf{x}_{i:i+h-1} + b)$$

- And picks the max as a feature for the CNN

$$\mathbf{c} = [c_1, c_2, \ldots, c_{n-h+1}] \quad \hat{c} = max\{\mathbf{c}\}$$



Yoon Kim - Convolutional Neural Networks for Sentence Classification

CNN for text classification

Many variations of the model [1]

- use existing vectors as input (CNN-static)
- learn vectors for the specific classification task through backpropagation (CNN-rand)
- Modify existing vectors for the specific task through backpropagation(CNN-non-static)

[1] Y. Kim, Convolutional Neural Networks for Sentence Classification, EMNLP 2014

CNN for text classification

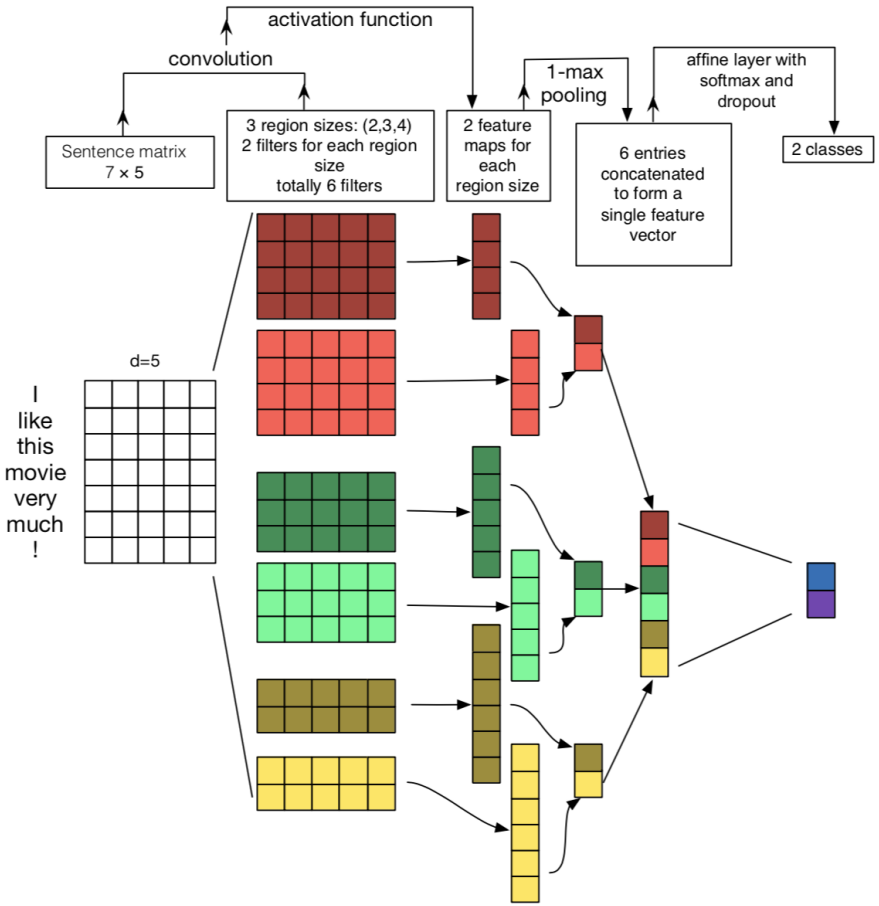
- Combine multiple word embeddings
- Each set of vectors is treated as a ‘channel’
- Filters applied to all channels
- Gradients are back-propagated only through one of the channels
- Fine-tunes one set of vectors while keeping the other static

CNN for text classification

Model	MR	SST-1	SST-2	Subj	TREC	CR	MPQA
CNN-rand	76.1	45.0	82.7	89.6	91.2	79.8	83.4
CNN-static	81.0	45.5	86.8	93.0	92.8	84.7	**89.6**
CNN-non-static	**81.5**	48.0	87.2	93.4	93.6	84.3	89.5
CNN-multichannel	81.1	47.4	**88.1**	93.2	92.2	**85.0**	89.4
RAE (Socher et al., 2011)	77.7	43.2	82.4	–	–	–	86.4
MV-RNN (Socher et al., 2012)	79.0	44.4	82.9	–	–	–	–
RNTN (Socher et al., 2013)	–	45.7	85.4	–	–	–	–
DCNN (Kalchbrenner et al., 2014)	–	48.5	86.8	–	93.0	–	–
Paragraph-Vec (Le and Mikolov, 2014)	–	**48.7**	87.8	–	–	–	–
CCAE (Hermann and Blunsom, 2013)	77.8	–	–	–	–	–	87.2
Sent-Parser (Dong et al., 2014)	79.5	–	–	–	–	–	86.3
NBSVM (Wang and Manning, 2012)	79.4	–	–	93.2	–	81.8	86.3
MNB (Wang and Manning, 2012)	79.0	–	–	**93.6**	–	80.0	86.3
G-Dropout (Wang and Manning, 2013)	79.0	–	–	93.4	–	82.1	86.1
F-Dropout (Wang and Manning, 2013)	79.1	–	–	**93.6**	–	81.9	86.3
Tree-CRF (Nakagawa et al., 2010)	77.3	–	–	–	–	81.4	86.1
CRF-PR (Yang and Cardie, 2014)	–	–	–	–	–	82.7	–
SVM_S (Silva et al., 2011)	–	–	–	–	**95.0**	–	–

Accuracy scores (Kim et al vs others)

CNN for text classification



- Data (text) only 1st column of input
- Rest of each row: embedding (in images 2D+RGB dimension)
- Filters of different sizes (4x5, 3x5 etc.)
  - Each size captures different features (need $\sim 10^2$ filters/size)
- Feature maps:
  - As many as the times filter fits on data matrix
- Max pooling maintains the “best features”
- Global feature map $\Rightarrow$ classification via softmax

[1] Zhang, Ye, and Byron Wallace. "A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification." arXiv preprint arXiv:1510.03820 (2015).

CNN architecture for document classification

T-SNE visualization

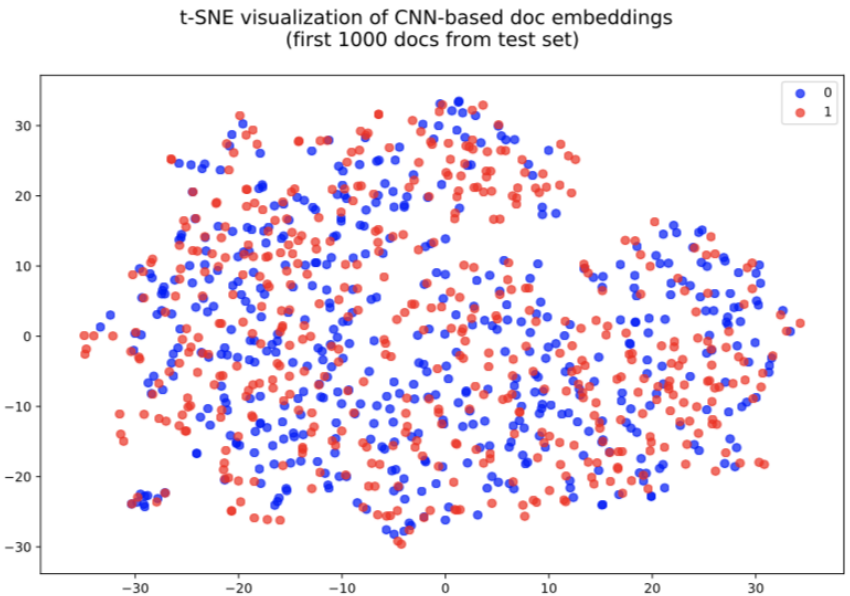


Figure 2: Doc embeddings before training.

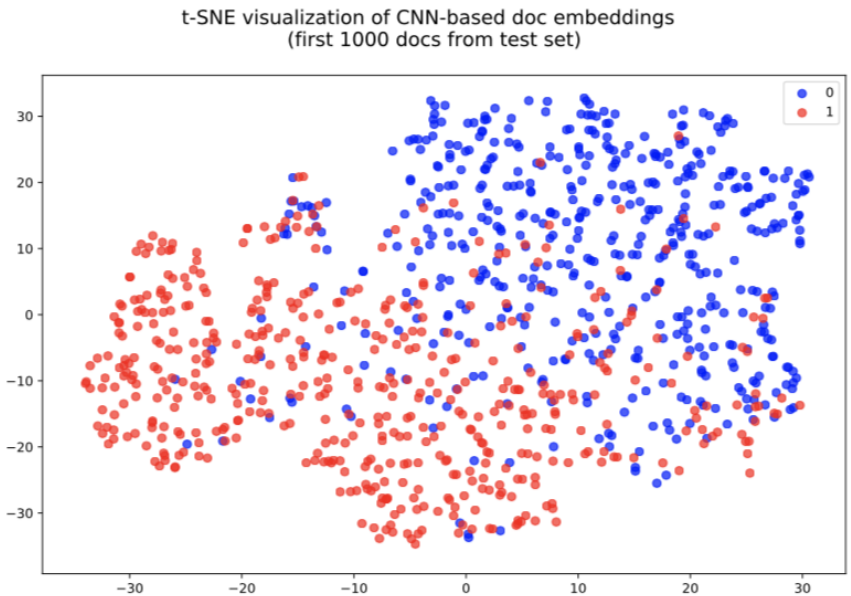


Figure 3: Doc embeddings after 2 epochs.

CNN architecture for document classification - Saliency maps

Saliency maps

- Words most related to changing the doc classification
- A in R^{sxd}, s:# sentence words, d:size of embeddings

$$saliency(a) = \left| \frac{\partial(CNN)}{\partial a} | a \right|$$

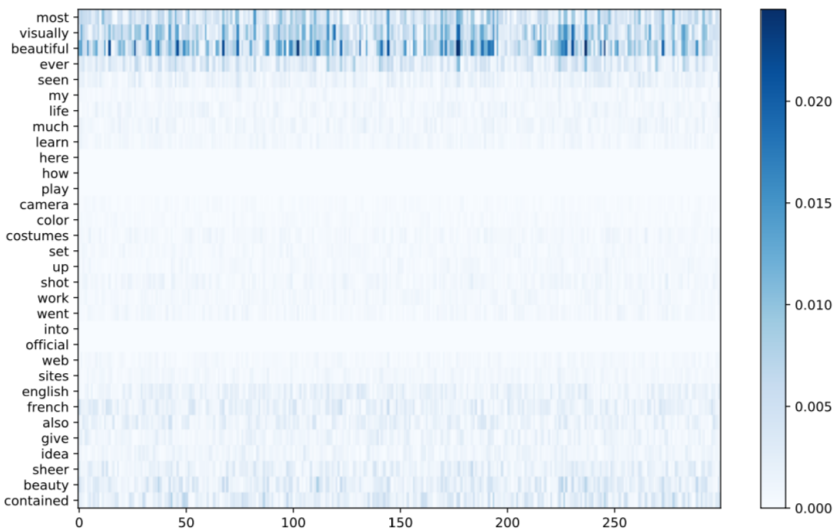


Figure 4: Saliency map for document 1 of the IMDB test set (true label: positive)

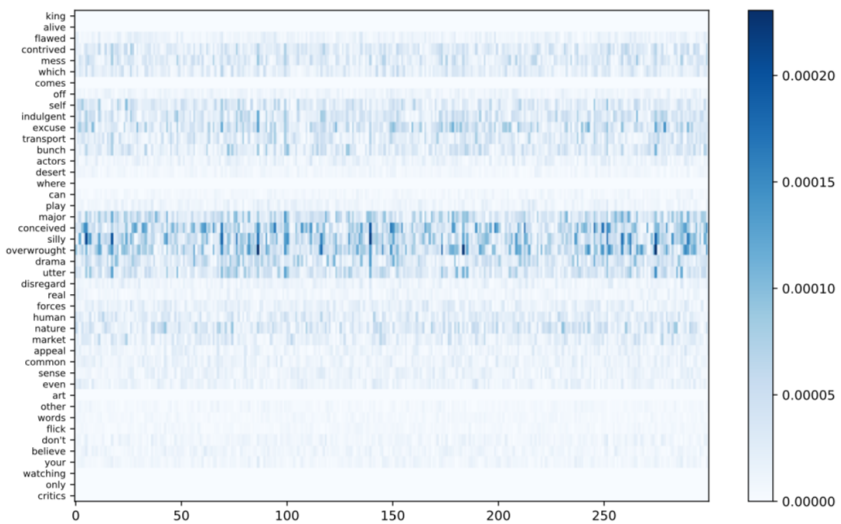


Figure 5: Saliency map for document 15 of the IMDB test set (true label: negative)